Practical Assignment 1

Minnie & Fran

Part 1 Dataset, Model Overview and Development

Dataset Description

- We used the **CIFAR-10 dataset**, which contains 10 classes: `['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']`.
- Each image is **RGB** with an original resolution of **32x32**. The dataset was split as follows:

`train size: 40000,val size: 10000,test size: 10000`

What we did for Preprocessing

- **Data augmentation:** Because the training size is relatively small, the model may easily overfit.
- **Normalize :** so that the model trains more stably and efficiently, by having all images on the same value scale.

Goal

Improve the base architecture by achieving over 82% accuracy without overfitting

Working Method

- Checkpoints in Google Drive
- Data division
- Define architecture
- Sanity Check
- Define hyperparameters
- Train model and safe
- Metrics (mCA/F1/Precision/Recall + Confusion Matrix) with scikit-learn

We did this for each combination of the model by changing hyperparameters.

Detailed analysis of the results obtained using four different hyperparameters.

Finally, we redraw the metrics of the best model

1.2 Results of 4 hyperparameters combinations

Evaluation of All Models (Test Set)

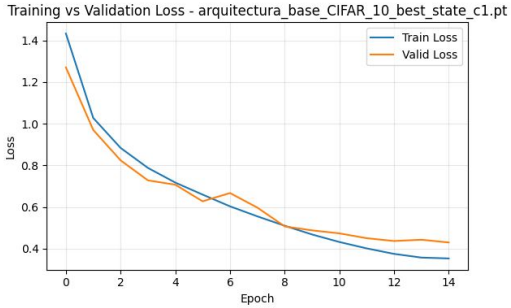
Model	Accuracy	Precision	Recall	F1-score	mCA
Model1 (arquitectura_base_CIFAR_10_best_state_c1.pt)	0.8935	0.8616	0.8618	0.8616	0.8618
Model2 (arquitectura_base_CIFAR_10_best_state_c2.pt)	0.7781	0.8164	0.8077	0.8091	0.8077
Model3 (arquitectura_base_CIFAR_10_best_state_c3.pt)	0.8496	0.8323	0.8332	0.8325	0.8332
Model4 (arquitectura_base_CIFAR_10_best_state_c4.pt)	0.8367	0.8272	0.8250	0.8239	0.8250

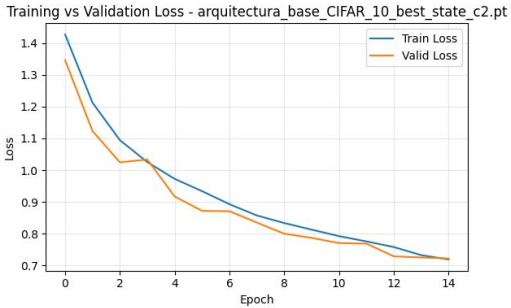
Models

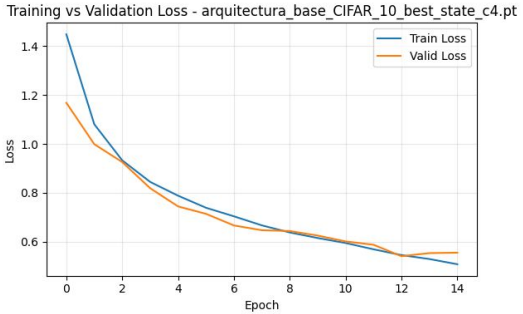
Model	Optimizer	Initial LR	Weights	Scheduler	Batch size	Dropout	Augmentation
Model1 (arquitectura_base_CIFAR_10_best_state_c1.pt)	Adam	5e-3	He + bias=0	Cosine	64	0.30	RandomCrop + RandomHorizontalFflip + ColorJitter
Model2 (arquitectura_base_CIFAR_10_best_state_c2.pt)	Adam	5e-3	He + bias=0	StepLR	64	0.35	RandomCrop + RandomHorizontalFflip + ColorJitter + RandomRotation + RandomFrasing
Model3 (arquitectura_base_CIFAR_10_best_state_c3.pt)	Adam	8e-4	He + bias=0	Cosine	128	0.25	RandomCrop + RandomHorizontalFflip + ColorJitter
Model4 (arquitectura_base_CIFAR_10_best_state_c4.pt)	SGD	0.03	He + bias=0	StepLR	64	0.25	RandomCrop + RandomHorizontalFflip + ColorJitter

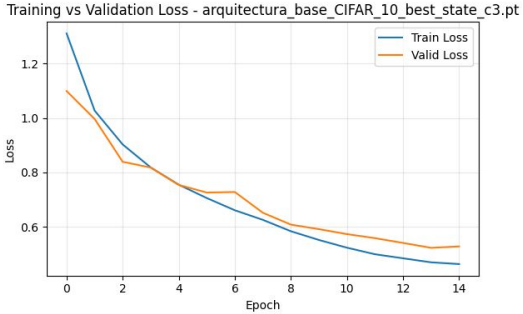
1.2 Results of 4 hyperparameters combinations

Loss curves

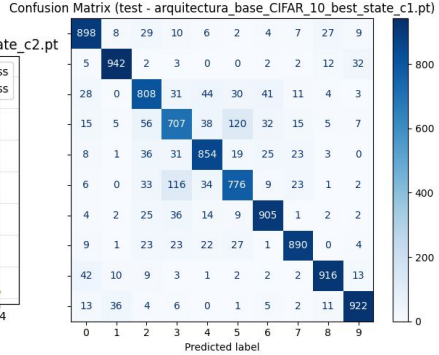


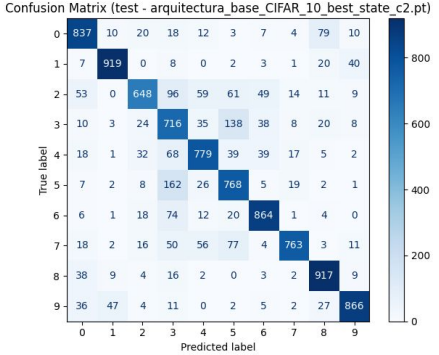


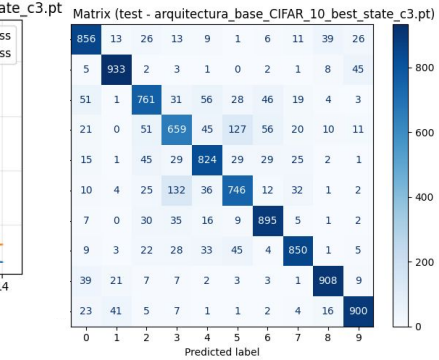


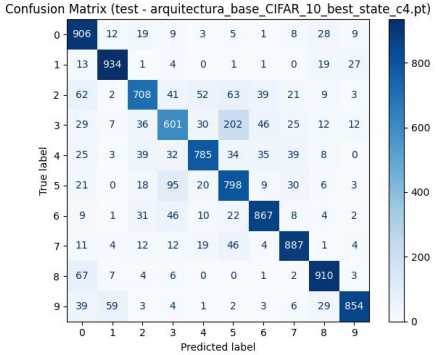


Confusion Matrix









1.3 CNN arquitecture

After considering all possible implementations to improve the base architecture, we have opted for this one.

Three convolutional blocks (64–128–256): progressively increase the number of filters to capture low-, mid-, and high-level visual features.

3×3 kernels: small receptive fields that effectively detect local patterns like edges, textures, and shapes.

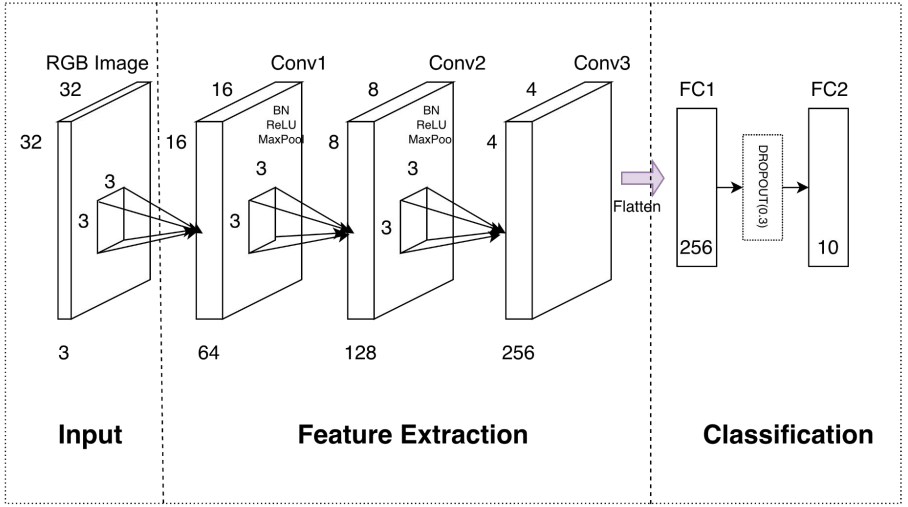
Batch Normalization after each convolution: stabilizes learning, accelerates convergence, and helps prevent overfitting (all data on the same scale)

ReLU activation: introduces non-linearity, allowing the network to learn complex relationships (the most commonly used today)

MaxPooling after each block: performs spatial downsampling, reducing the image resolution while keeping the most relevant features.

Flatten + Fully Connected layers: transform spatial features into class-level representations for final classification.

Dropout only in the head: applied where most parameters are concentrated to prevent overfitting, which is sufficient given CIFAR-10's small size.



- 3 convolutional blocks: 64–128–256 filters, 3×3 kernels.
- Each block: Conv → BatchNorm → ReLU → MaxPool(2×2).
- Feature maps: [3×32×32] → [64×16×16] → [128×8×8] → [256×4×4].
- Classification head: Flatten → FC(4096→256) → Dropout(0.30) → FC(256→10).
- All layers are trainable (no frozen backbones).

1.4 Comparative Analysis

Model 1 (Adam + Cosine + batch 64 + dropout 0.30)

- It was the model with the best overall performance: Accuracy = 89.35%, mCA = 86.18%, F1 = 86.16%.
- The combination of the Adam optimizer with the Cosine scheduler allowed for smooth and stable convergence.
- Moderate dropout (0.3) and data augmentation techniques (RandomCrop, Flip, ColorJitter) prevented overfitting, maintaining good generalization.
- The loss curves show a steady decline with no pronounced separation between training and validation, indicating a balance between learning and regularization.
- Confusion matrix shows strong diagonal dominance, the model correctly identifies most classes with low confusion. However, it often fails like everyone else in classifying both cats and dogs.

Model 2 (Adam + StepLR + batch 64 + dropout 0.35 + data augmentation)

- Additional augmentation techniques (rotations, Random Erasing) and a higher dropout rate (0.35) were applied.
- Although these measures aim to increase robustness, in this case they led to underfitting.
- The model failed to correctly learn the main characteristics of the dataset.
- The learning rate was reduced too quickly by StepLR, slowing down learning.
- Result: lower performance (mCA = 80.77%) and flatter loss curves, with little improvement in validation

Model 3 (Adam + Cosine + batch 128 + dropout 0.25)

- It obtained intermediate results (mCA = 83.32%)
- Increasing the batch size to 128 reduced gradient variability, which improved stability but slightly worsened generalization ability.
- The lowest dropout (0.25) offered faster training but favored slight overfitting towards the end.

Model 4 (SGD + StepLR + batch 64 + dropout 0.25)

- The use of SGD with a high learning rate (0.03) and StepLR produced slower and more unstable convergence.
- Although it used the same augmentation techniques as Model 1, the optimizer failed to fine-tune the weights, resulting in less efficient learning.

CONCLUSION

- The Adam optimizer showed more stable and effective behavior than SGD for this type of CNN and dataset.
- The Cosine Scheduler favored more gradual convergence and better final results than StepLR.
- Basic augmentation techniques (cropping, horizontal flipping, color jitter) were sufficient; adding excessive rotations or erasing reduced performance.
- No model showed strong signs of overfitting, indicating correct use of dropout and regularization.
- Model 1 achieved the best balance between accuracy and generalization, meeting the target of 82% mean Class Accuracy.

Part 2 Dataset and Model Overview

Dataset Description

- We used the **STL-10 dataset**, which contains 10 classes: `['airplane', 'bird', 'car', 'cat', 'deer', 'dog', 'horse', 'monkey', 'ship', 'truck']`.
- Each image is **RGB** with an original resolution of **96×96**. The dataset was split as follows:

```
train size: 4000,val size: 1000,test size: 8000
```

- Model Selection

Given limited computational resources (Colab), we selected **ResNet-18 and MobileNetV3 Small**

What we did for Preprocessing

- **Data augmentation(RandomHorizontalFlip):** Because the training size is relatively small, the model may easily overfit.
- **Resize and Normalize :** Since pretrained models on ImageNet expect input size of 224×224, we applied the same preprocessing pipeline to both pretrained and scratch training

Unified Training Protocol

- **Training from scratch**:using the same hyperparameter settings.
- **Training with pretrained weights**: using pretrained weights provided by torchvision

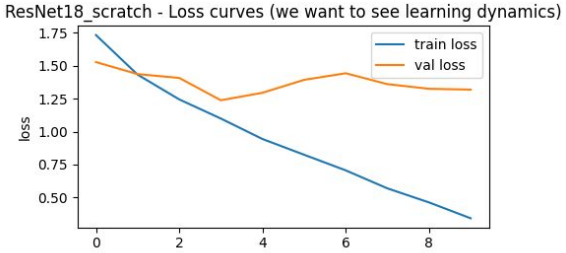
Hyperparameter	Setting
Optimizer	Adam
Learning Rate	1e-4
Batch Size	16
Epochs	10
Loss Function	CrossEntropyLoss
best trained models	Best checkpoint by validation accuracy

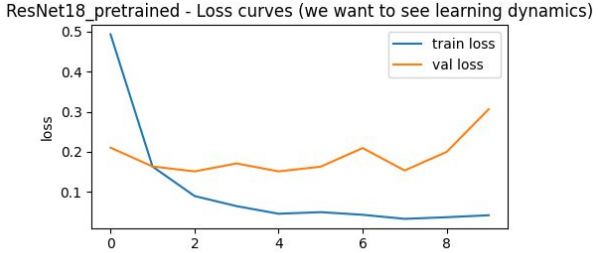
2.2 Results:Scratch,Pretrained and Middle Fusion

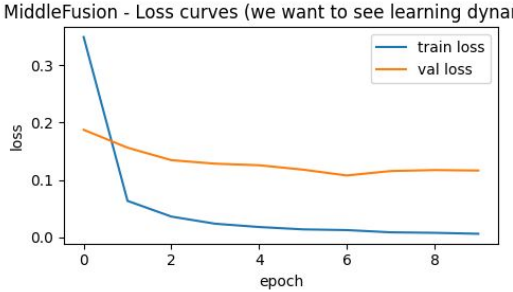
Evaluation of All Models (Test Set)

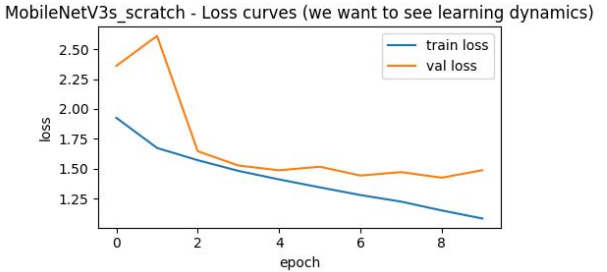
Model	Accuracy	Precision	Recall	F1-score	mCA
ResNet18 (Scratch)	0.612	0.634	0.612	0.614	0.612
MobileNetV3-small (Scratch)	0.497	0.507	0.497	0.486	0.497
ResNet18 (Pretrained)	0.944	0.946	0.944	0.944	0.944
MobileNetV3-small (Pretrained)	0.936	0.937	0.936	0.936	0.936
Middle Fusion (ResNet18 + MobileNetV3-small) BEST	**0.961**	**0.96**	**0.96**	**0.96**	**0.96**

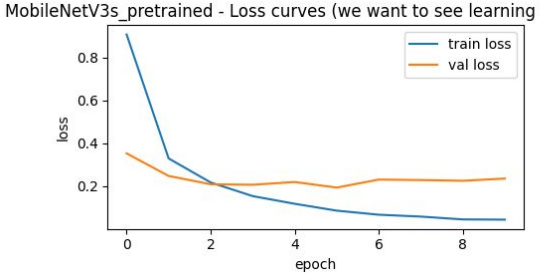
Loss curves







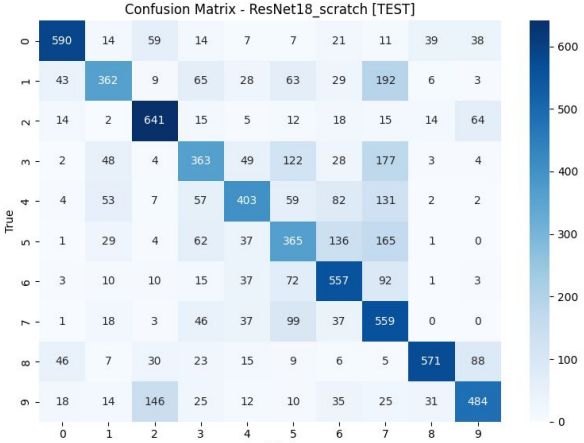


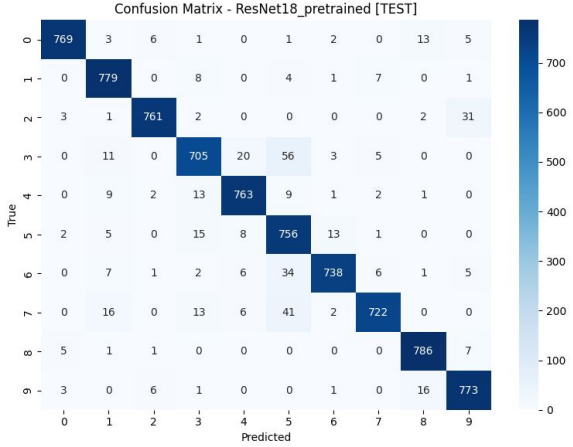


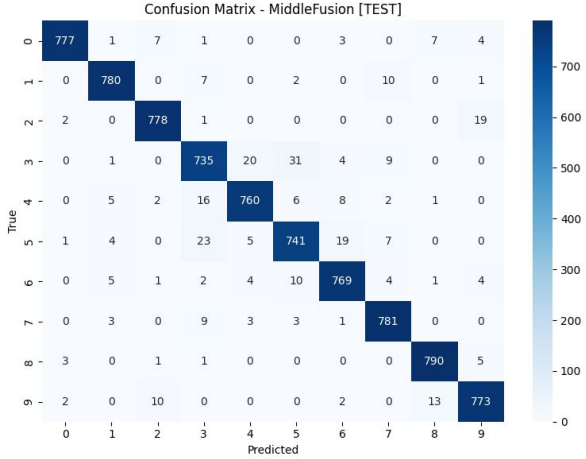
1.For the models trained from scratch, both ResNet18 and MobileNetV3-small started with high training and validation losses, and their curves decreased slowly.
2.The pretrained models converged much faster.Both ResNet18 and MobileNetV3-small (pretrained) quickly reached low training and validation losses within the first few epochs.
3.The Middle Fusion model displayed the lowest and most stable losses. Its validation curve remained almost flat after the second epoch, which shows excellent generalization.

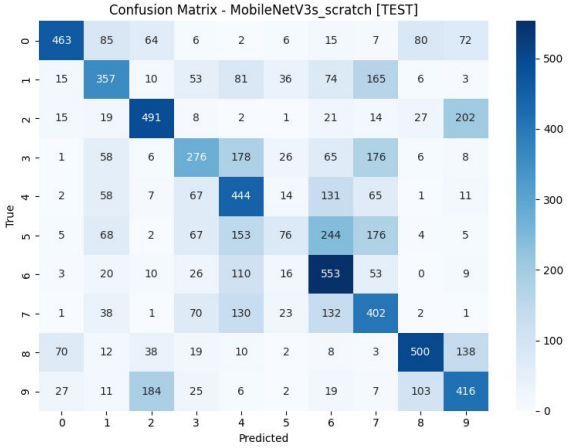
2.2 Results:Scratch,Pretrained and Middle Fusion

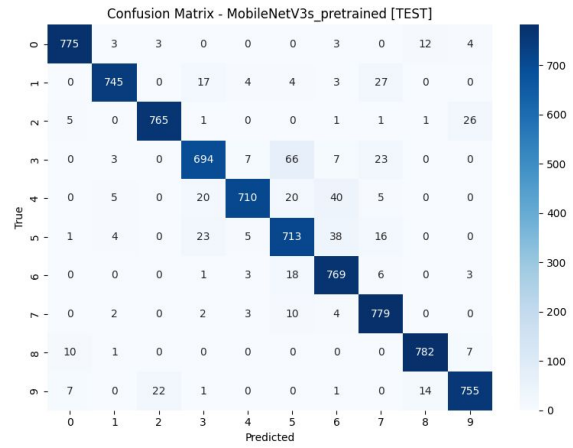
Confusion Matrix









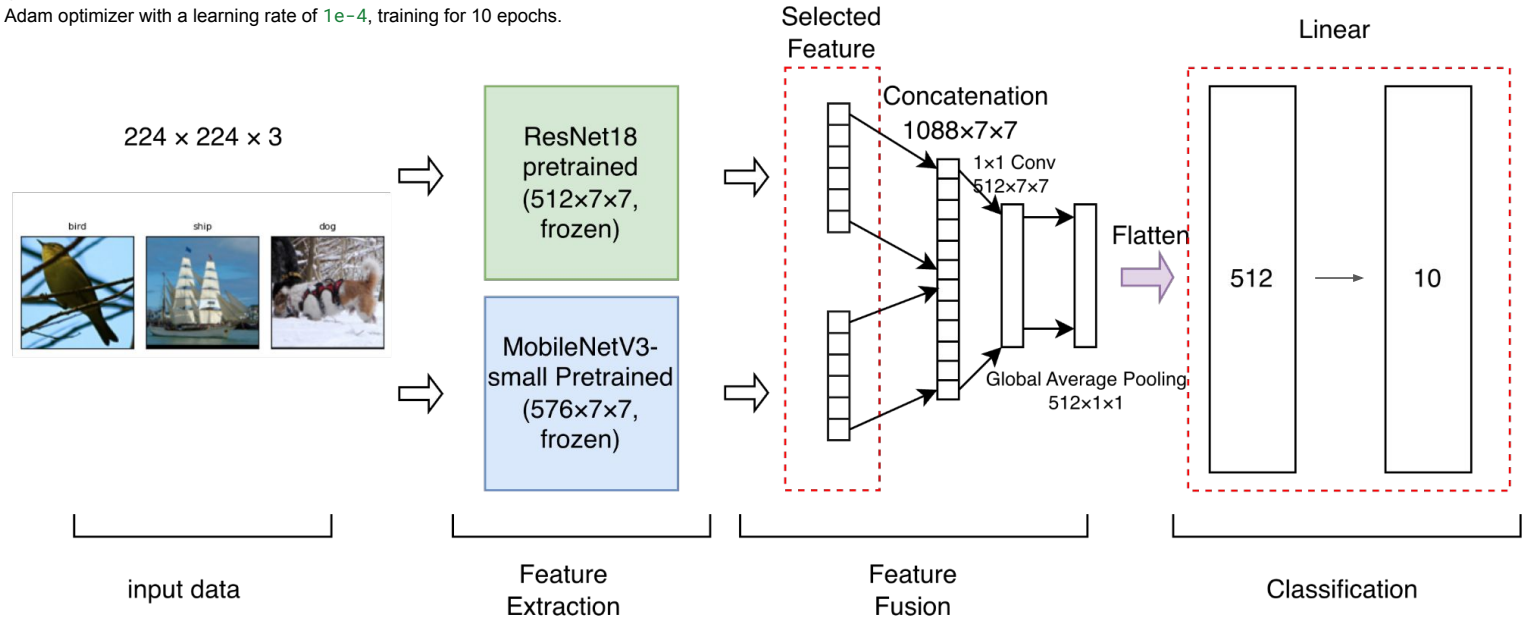


1.Models trained from scratch made many classification mistakes and often confused similar classes.
2.Pretrained models performed much better, with cleaner diagonals and fewer errors.
3.The Middle Fusion model shows the clearest and most dominant diagonal line among all five matrices.
Misclassifications are minimal, and no single class shows consistent confusion with another.

2.3 Middle Fusion Method

After training four individual models (ResNet18 and MobileNetV3-small, both from scratch and with pretrained weights), we selected the best performing versions to build a Middle Fusion network.The Middle Fusion model then works as follows:

1. Feature Extraction:Both frozen backbones process the same input image.
   ResNet18 produces a feature map of shape `[512, 7, 7]`.
   MobileNetV3-small produces `[576, 7, 7]`.
2. Feature Fusion (Concatenation):The two feature maps are concatenated along the channel dimension, forming a combined tensor `[1088,7,7]`.
3. Channel Compression: A 1×1 convolution reduces the number of channels from 1088 to 512.This step prevents the model from becoming too heavy and helps focus on the most informative features.
4. Global Average Pooling and Classification:The fused features are passed through a Global Average Pooling layer to summarize each channel into one value (`[512, 1, 1]`).Then, a fully connected layer (Linear 512→10) performs the final classification across the 10 STL-10 categories.
5. During training, the two pretrained networks remain frozen.
6. We used the Adam optimizer with a learning rate of `1e-4`, training for 10 epochs.



2.4 Comparative Analysis

1. Scratch vs Pretrained Models.

When trained from scratch, both ResNet18 and MobileNetV3-small showed slow convergence on the STL-10 dataset.
This is mainly because STL-10 contains a limited number of images, so the models had to learn all visual features from zero, such as edges, colors, and textures. ResNet18 eventually reached a validation accuracy of about **0.61**, while MobileNetV3-small performed worse with only **0.50** accuracy. This indicates that small datasets are not enough for deep networks to learn rich features effectively.

In contrast, the pretrained models converged much faster and more stably.By using weights learned from ImageNet, both networks started with strong feature representations and quickly reached validation accuracy above **0.93** within the first few epochs.This clearly shows the advantage of **transfer learning**, as pretrained models can reuse previously learned general image features instead of learning everything from scratch.

2. ResNet18 and MobileNetV3-small

Although both ResNet18 and MobileNetV3-small are convolutional neural networks, they follow different design ideas.ResNet18 uses residual connections, meaning that each residual block has a “shortcut” that adds the input back to the output.This design helps prevent the vanishing gradient problem and allows the network to go deeper, learning more complex and detailed features. As a result, ResNet18 achieves higher accuracy but requires more computation and memory resources.MobileNetV3-small is designed for efficiency.It uses depthwise separable convolutions.This makes the model much faster and smaller, ideal for mobile and embedded devices.However, its simplified structure means it cannot learn as rich and detailed features as ResNet18, which explains its slightly lower accuracy in both training modes.

3. Performance of the Middle Fusion model

The Middle Fusion model achieved the best results overall, with a test accuracy of 0.961,In this model, both pretrained feature extractors (ResNet18 and MobileNetV3-small) were frozen and used together.They processed the same input image independently. ResNet18 focused on *fine details* such as texture and edges.MobileNetV3-small focused on *global shapes* and object structure.This design allowed the fusion head to learn how to merge both detailed and global information.The confusion matrix of the fusion model showed a stronger diagonal pattern,meaning that the model made fewer mistakes across all ten categories.This improvement happens because the two backbones provide complementary information.ResNet brings detailed local understanding, while MobileNet offers global spatial context.By fusing them, the model gained a more balanced and comprehensive view of each image, which explains its higher overall performance.

4. Why regularization was not used in the Middle Fusion model

No explicit regularization techniques such as weight decay or dropout were used in the Middle Fusion model.Despite this, the model achieved strong generalization (Test Acc = 96.1%).both backbones were pretrained on ImageNet, providing strong prior knowledge. Also the backbone parameters were frozen, reducing the number of trainable weights.Therefore, the model did not require additional regularization to achieve stable and high performance.

However, in future work, adding light dropout (p=0.3) after the fusion layer may further enhance robustness against overfitting.